

DATA WAREHOUSE MODELING

Data Warehouse Modeling

- Data warehouses and OLAP tools are based on a **multidimensional data model**
- This model views data in the form of a **data cube**

Data Cube: A Multidimensional Data Model

- A data cube allows data to be modeled and viewed in multiple dimensions
- It is defined by dimensions and facts

Dimensions

- Perspectives or entities with respect to which an organization wants to keep records
- E.g.: *AllElectronics* may create a *sales* data warehouse in order to keep records of the store's sales with respect to the dimensions *time*, *item*, *branch*, and *location*
- These dimensions allow the store to keep track of things like monthly sales of items and the branches and locations at which the items were sold
- Each dimension may have a table associated with it, called a **dimension table**, which further describes the dimension
- E.g: dimension table for *item* may contain the attributes *item name*, *brand*, and *type*

Facts

- A multidimensional data model is typically organized around a central theme, such as *sales*
- This theme is represented by a fact table
- Facts are numeric measures/quantities by which we can analyze relationships between dimensions
- E.g: for a sales data warehouse facts could be *dollars sold* (sales amount in dollars), *units sold* (number of units sold), and *amount budgeted*
- fact table contains the names of the *facts*, or measures, as well as keys to each of the related dimension tables

- In data warehousing the data cube is n -dimensional

2-D View of Sales Data for *AllElectronics* According to *time* and *item*

<i>location</i> = "Vancouver"				
<i>time (quarter)</i>	<i>item (type)</i>			
	<i>home entertainment</i>	<i>computer</i>	<i>phone</i>	<i>security</i>
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Note: The sales are from branches located in the city of Vancouver. The measure displayed is *dollars_sold* (in thousands).

- In this 2-D representation, the sales for Vancouver are shown with respect to the
 - *time* dimension (organized in quarters) and the *item* dimension (organized according to the types of items sold).
 - The fact or measure displayed is *dollars sold* (in thousands)

- To view the sales data according to time and item, as well as location, for the cities Chicago, New York, Toronto, and Vancouver:

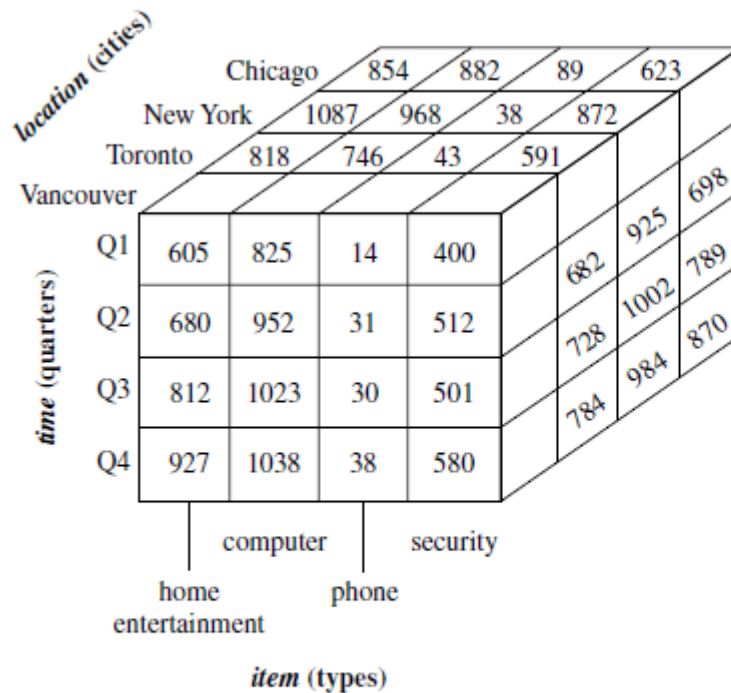
Table 4.3 3-D View of Sales Data for *AllElectronics* According to *time*, *item*, and *location*

<i>location</i> = "Chicago"					<i>location</i> = "New York"					<i>location</i> = "Toronto"					<i>location</i> = "Vancouver"				
<i>item</i>					<i>item</i>					<i>item</i>					<i>item</i>				
<i>home</i>					<i>home</i>					<i>home</i>					<i>home</i>				
<i>time</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>		<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>		<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	
Q1	854	882	89	623	1087	968	38	872		818	746	43	591		605	825	14	400	
Q2	943	890	64	698	1130	1024	41	925		894	769	52	682		680	952	31	512	
Q3	1032	924	59	789	1034	1048	45	1002		940	795	58	728		812	1023	30	501	
Q4	1129	992	63	870	1142	1091	54	984		978	864	59	784		927	1038	38	580	

Note: The measure displayed is *dollars_sold* (in thousands).

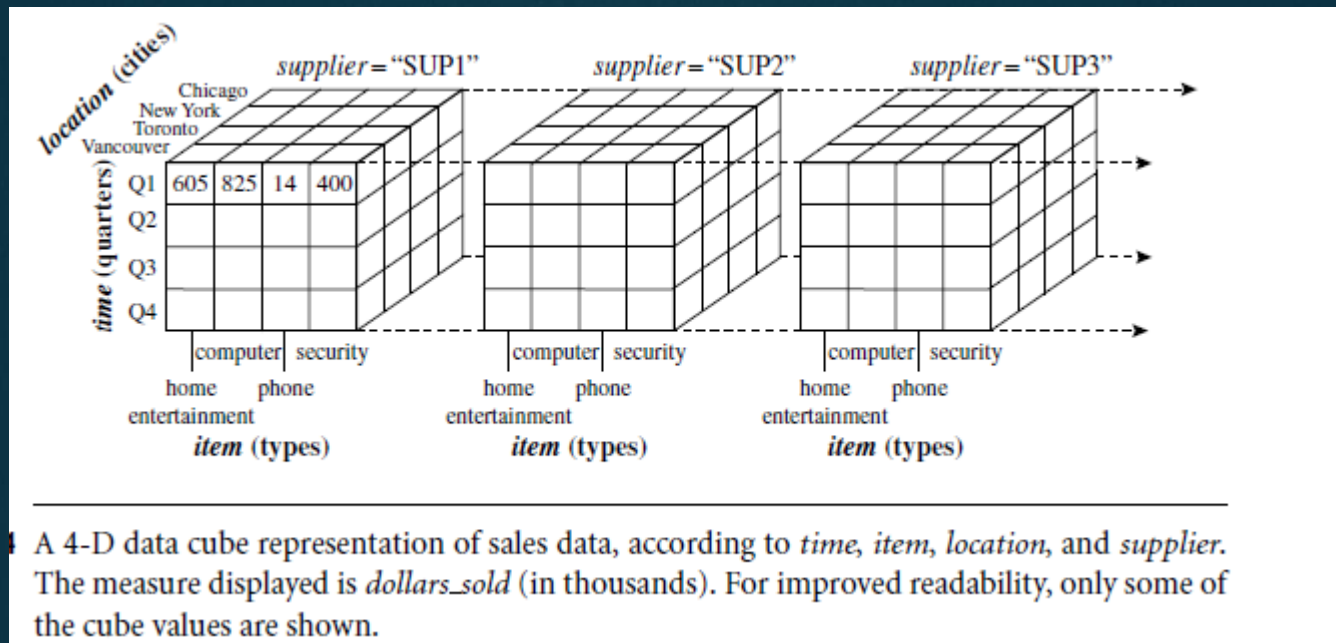
- The 3-D data in the table are represented as a series of 2-D tables

- Conceptually, we may also represent the same data in the form of a 3-D data cube

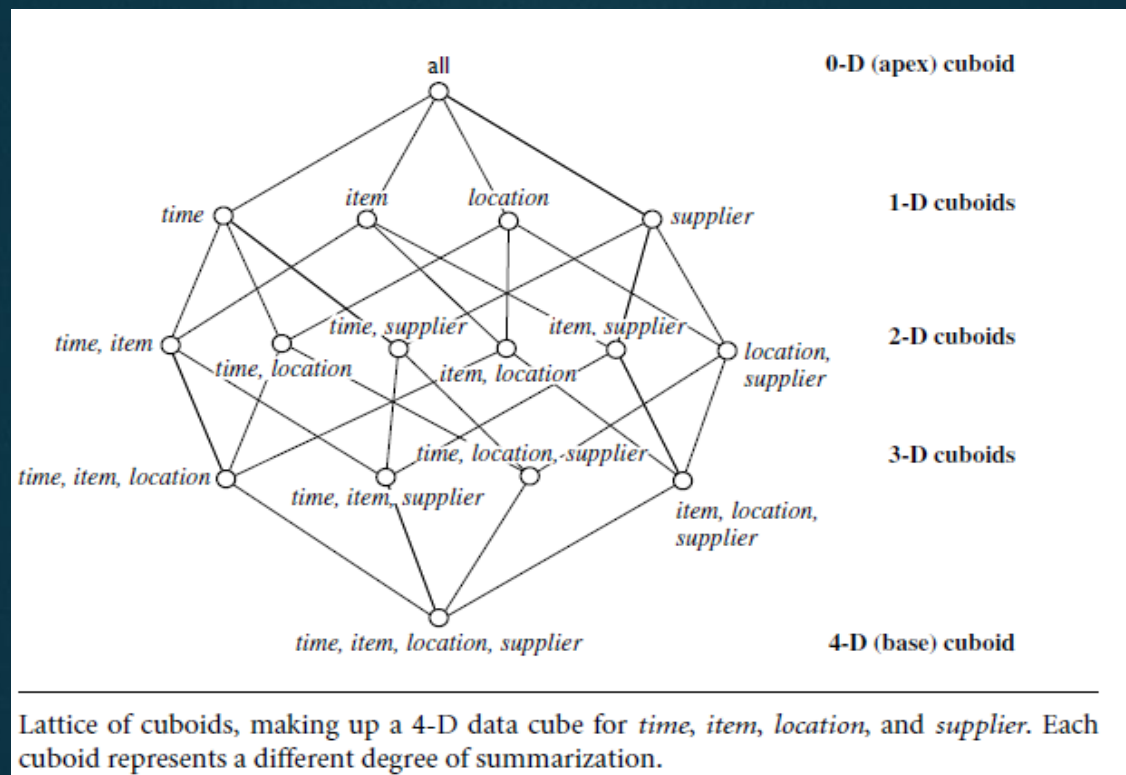


A 3-D data cube representation of the data in Table 4.3, according to *time*, *item*, and *location*. The measure displayed is *dollars_sold* (in thousands).

- To view sales data with an additional fourth dimension such as *supplier*:
- We can think of a 4-D cube as being a series of 3-D cubes



- Given a set of dimensions, we can generate a cuboid for each of the possible subsets of the given dimensions.
- The result would form a *lattice* of cuboids, each showing the data at a different level of summarization, or group-by.
- The lattice of cuboids is then referred to as a data cube



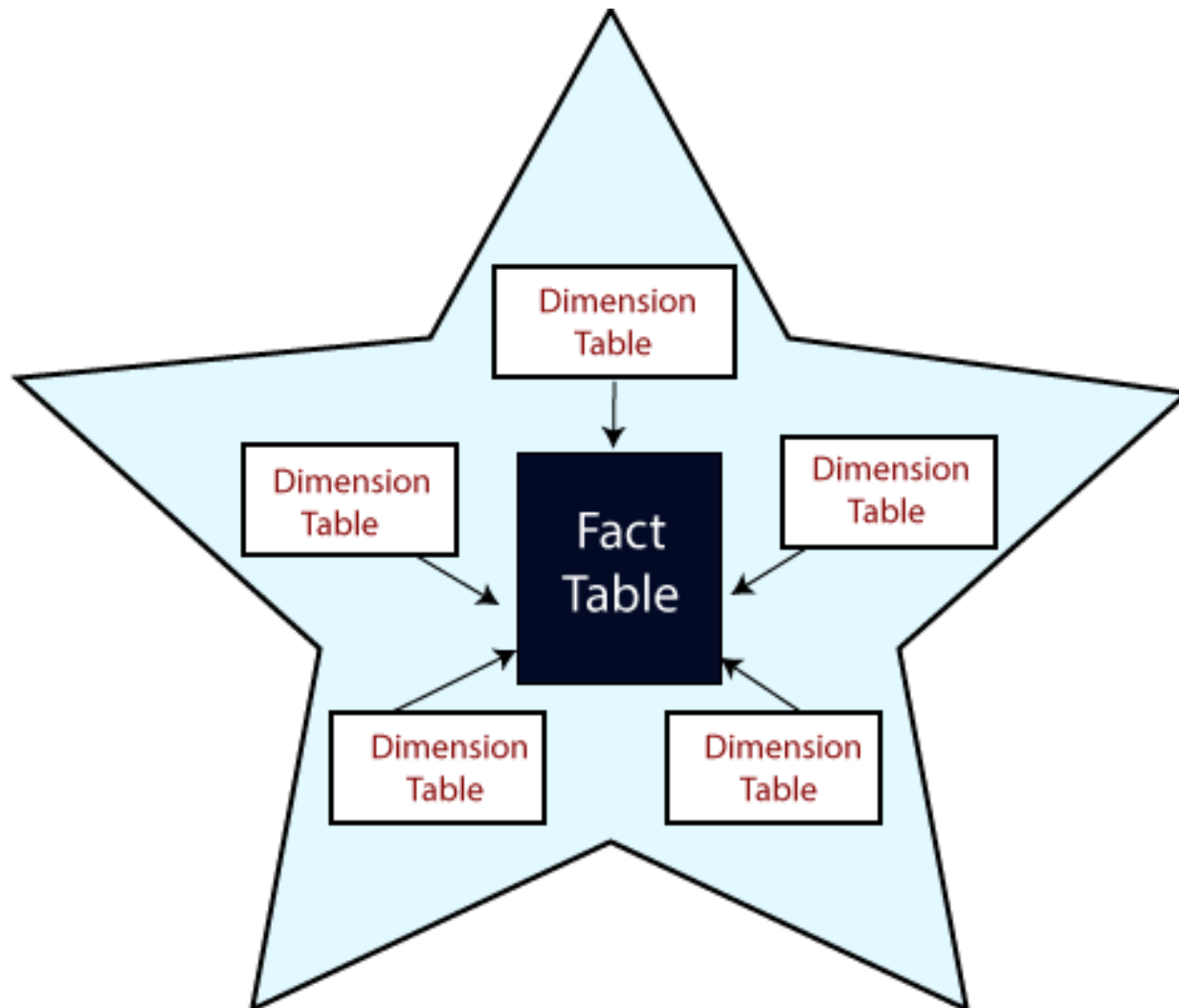
- The cuboid that holds the lowest level of summarization is called the **base cuboid**
- The 0-D cuboid, which holds the highest level of summarization, is called the **apex cuboid**.
- In our example, this is the total sales, or *dollars sold*, summarized over all four dimensions. The apex cuboid is typically denoted by *all*.

Schemas for Multidimensional Data Models

- A data warehouse requires a concise, subject-oriented schema that facilitates online data analysis
- The most popular data model for a data warehouse is a multidimensional model
- It exist in the form of a **star schema**, a **snowflake schema**, or a **fact constellation schema**

Star schema

- The most common modeling paradigm
- In this, the data warehouse contains
- A large central table (**fact table**) containing the bulk of the data, with no redundancy
- A set of smaller attendant tables (**dimension tables**), one for each dimension
- The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table
- Star schemas are denormalized

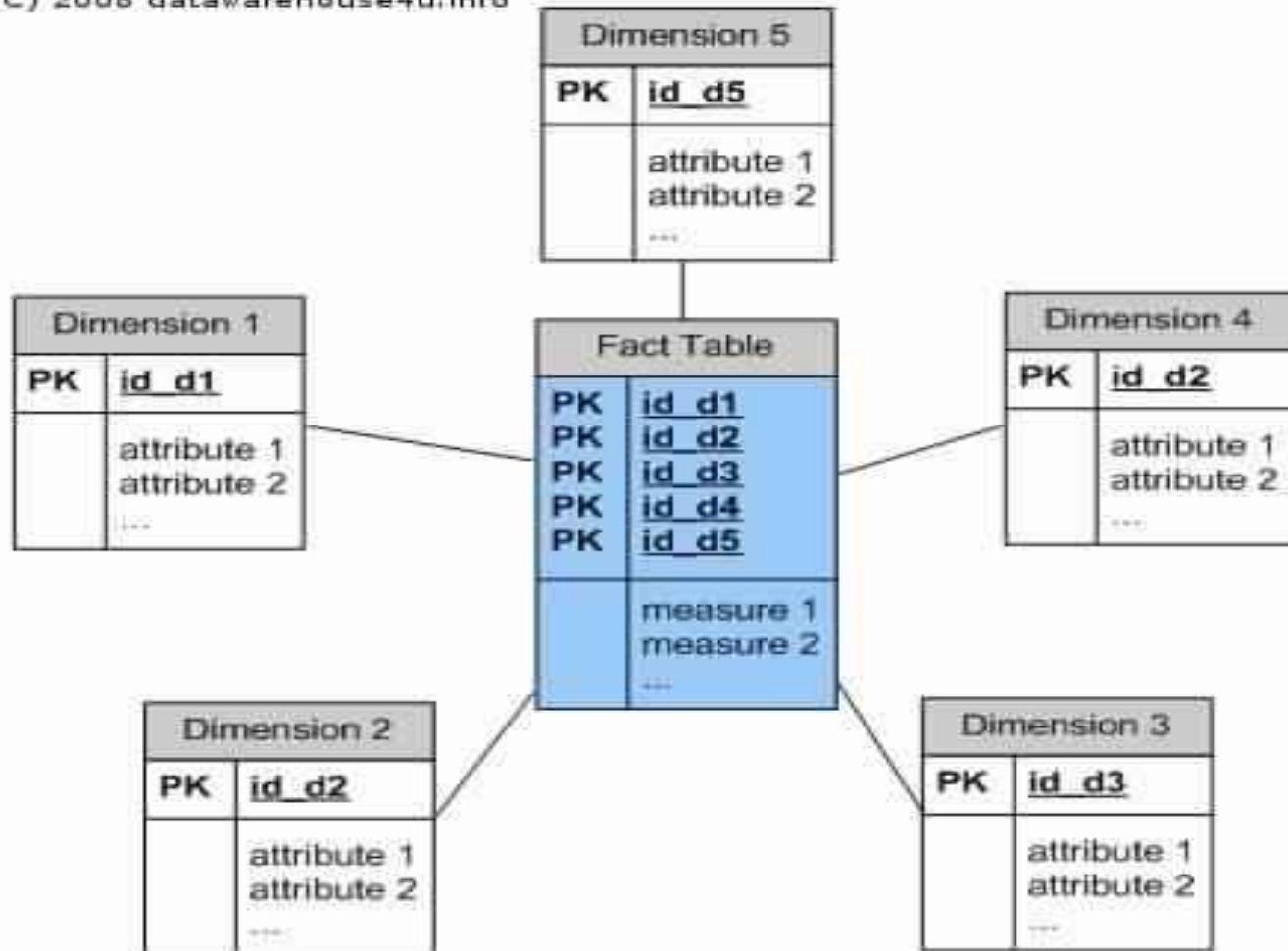


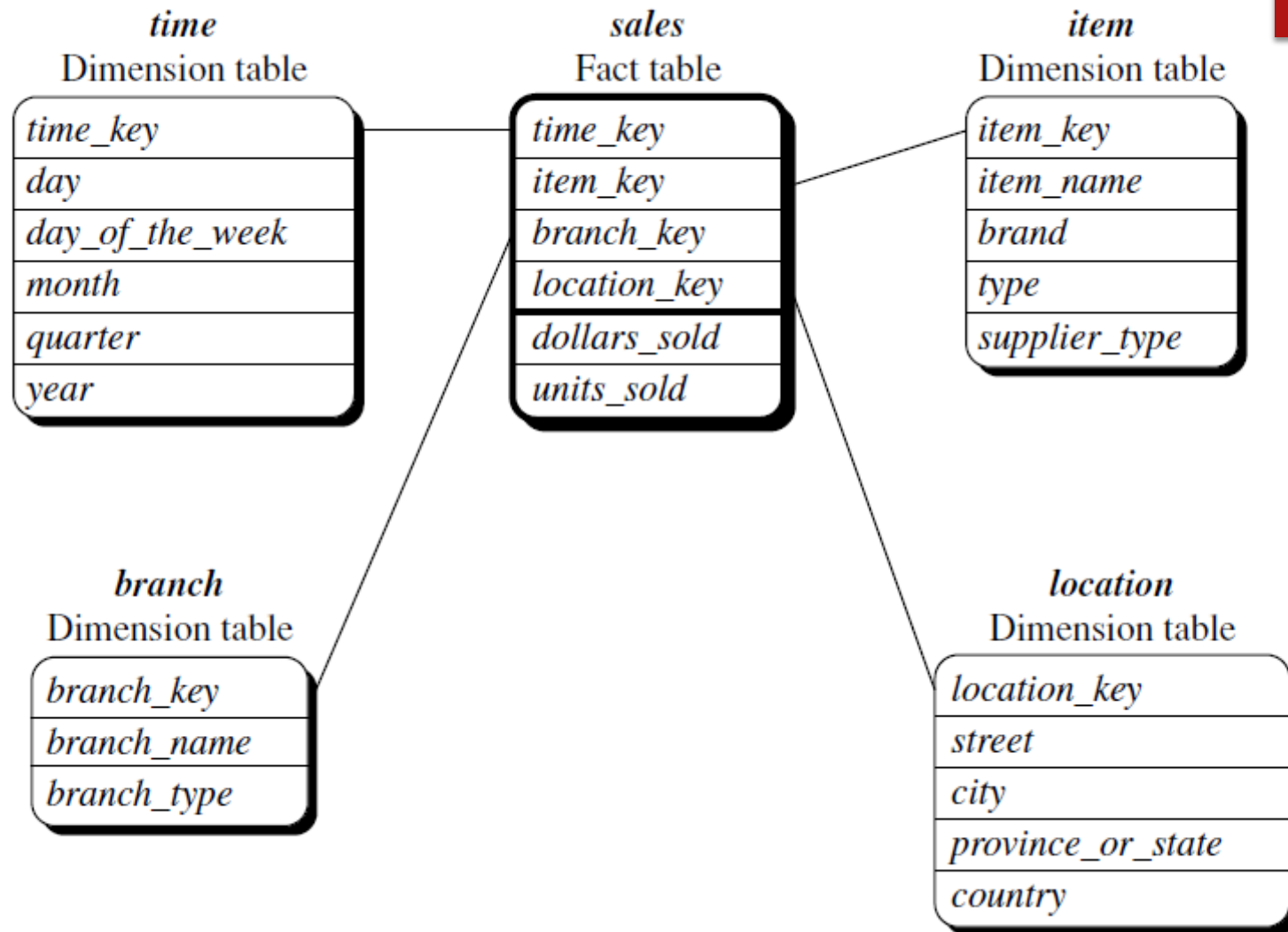
Star Schema

Star schema for *AllElectronics* sales

- Sales are considered along four dimensions: *time*, *item*, *branch*, and *location*
- The schema contains a central fact table for *sales* that contains keys to each of the four dimensions, along with two measures: *dollars sold* and *units sold*
- each dimension is represented by only one table, and each table contains a set of attributes
- Example: the *location* dimension table contains the attribute set {*location key*, *street*, *city*, *province_or_state*, *country*}

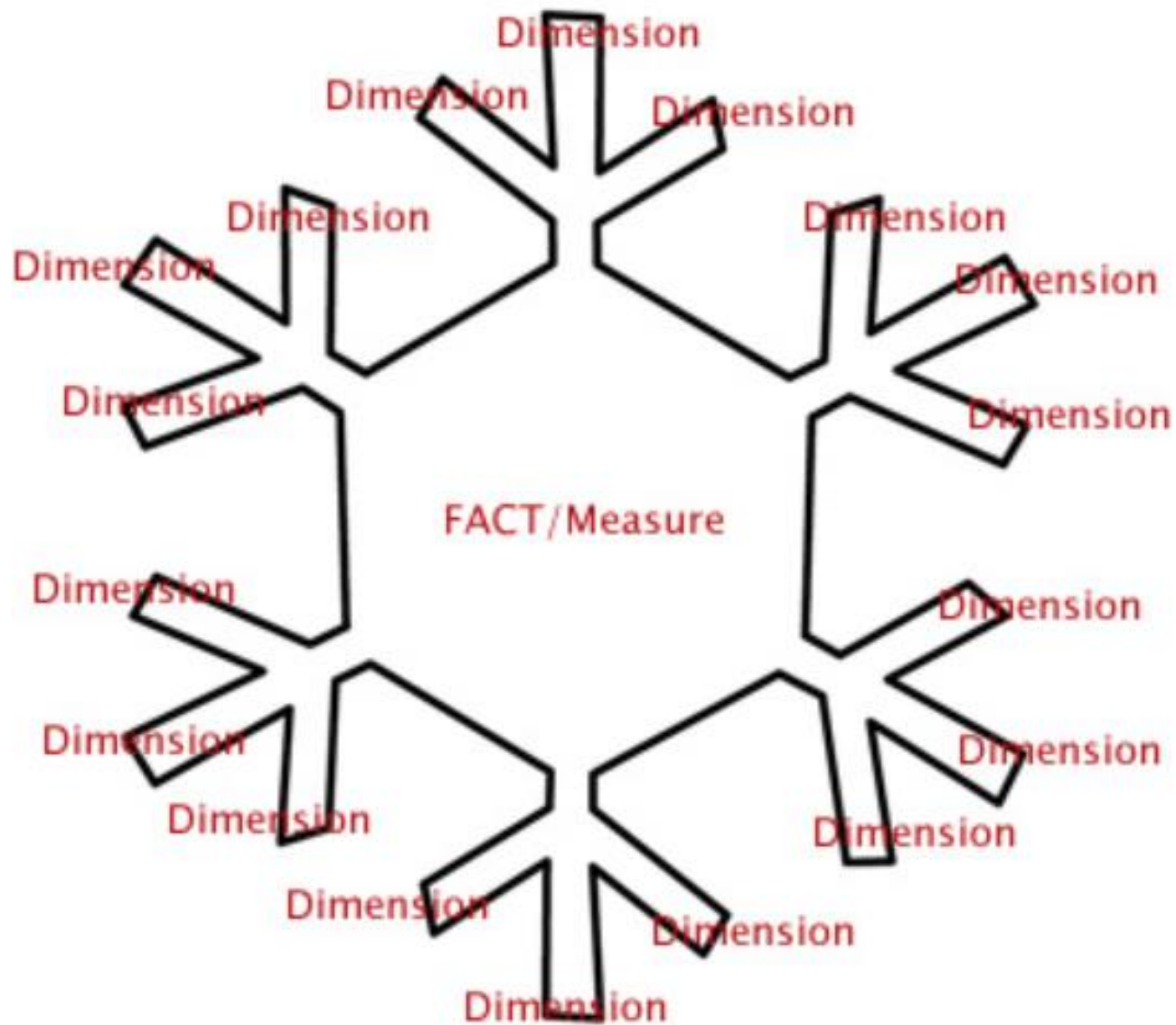
(C) 2008 datawarehouse4u.info





Snowflake schema

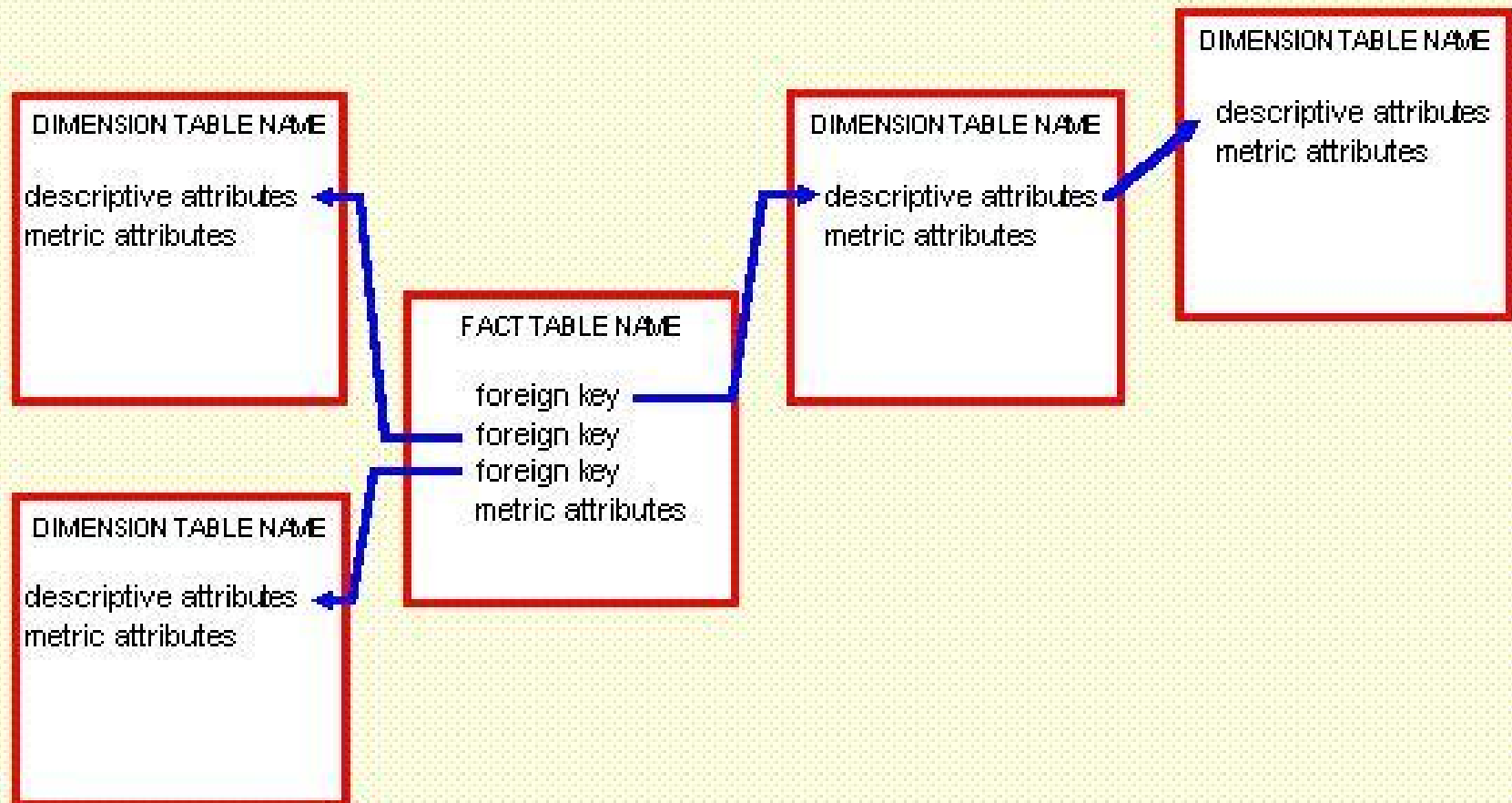
- The snowflake schema is a variant of the star schema model
- Here some dimension tables are *normalized*, thereby further splitting the data into additional tables
- The resulting schema graph forms a shape similar to a snowflake
- Dimension tables of the snowflake model are kept in normalized form to reduce redundancies
- Such tables are easy to maintain and saves storage space
- However, this space savings is negligible in comparison to the typical magnitude of the fact table
- Also, the snowflake structure can reduce the effectiveness of browsing, since more joins will be needed to execute a query => system performance may be adversely impacted
- Hence, although the snowflake schema reduces redundancy, it is not as popular as the star schema in data warehouse design

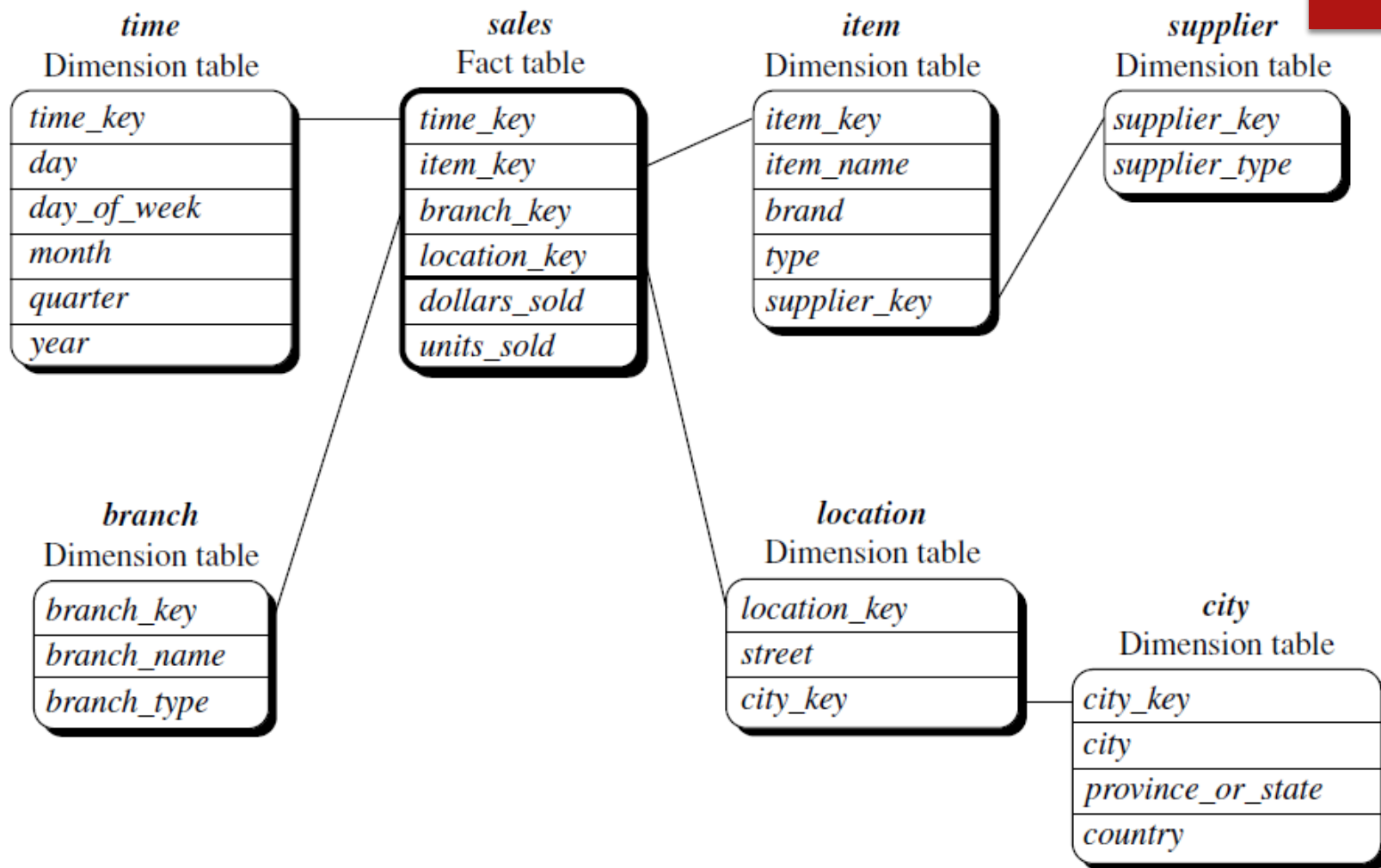


Example:

- The *sales* fact table is identical to that of the star schema
- The single dimension table for *item* in the star schema is normalized in the snowflake schema, resulting in new *item* and *supplier* tables
- For example, the *item* dimension table now contains the attributes *item key*, *item name*, *brand*, *type*, and *supplier key*, where *supplier key* is linked to the *supplier* dimension table, containing *supplier key* and *supplier type* information
- single dimension table for *location* in the star schema can be normalized into two new tables: *location* and *city*
- The *city key* in the new *location* table links to the *city* dimension.

Snowflake Schema Template



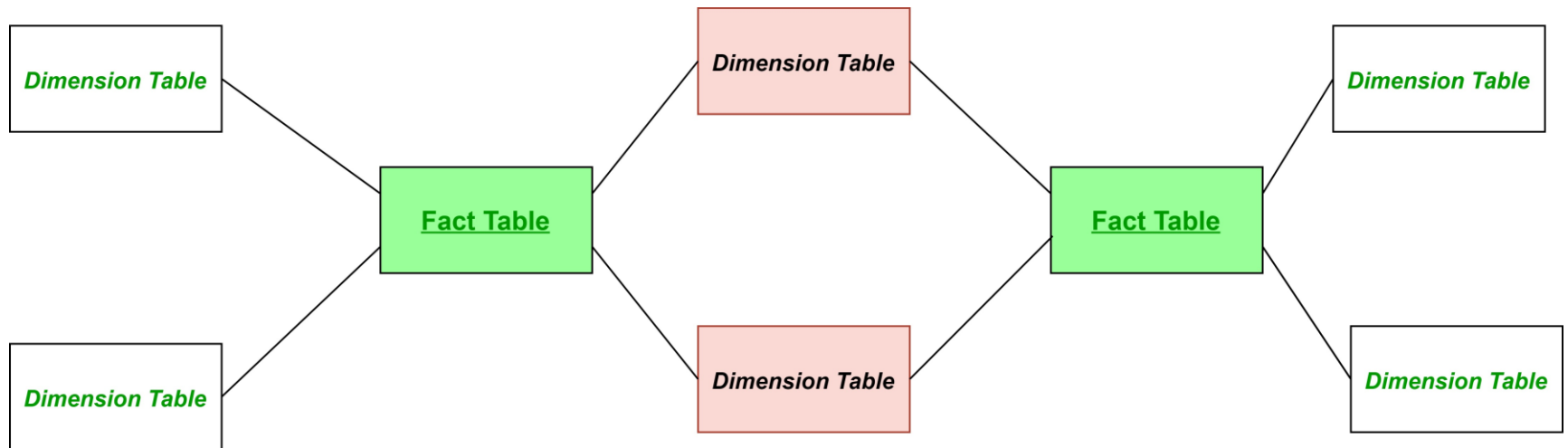


Difference between Snow Flake and Star Schema

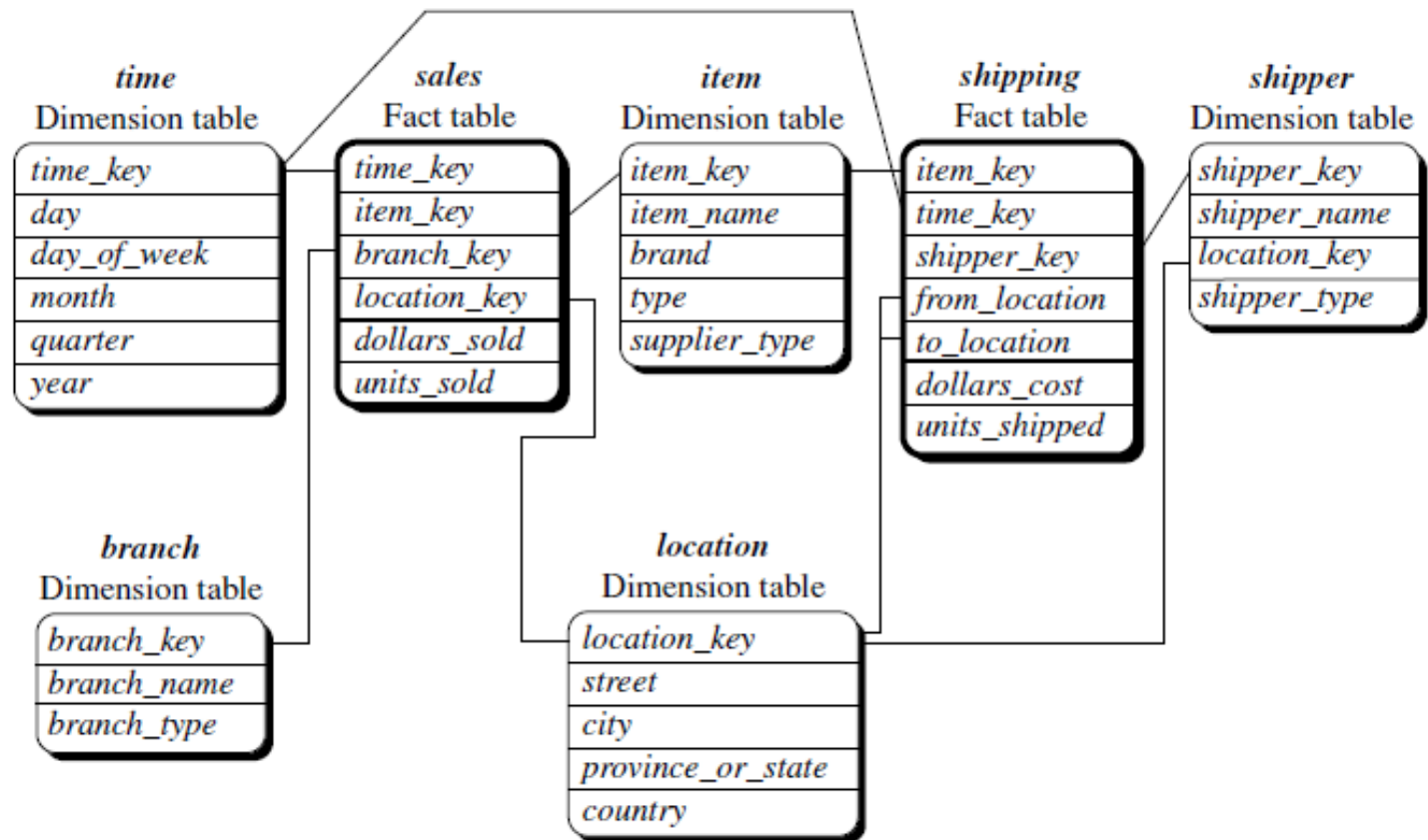
Star Schema	Snow Flake Schema
The star schema is the simplest data warehouse scheme.	Snowflake schema is a more complex data warehouse model than a star schema.
In star schema each of the dimensions is represented in a single table .It should not have any hierarchies between dims.	In snow flake schema at least one hierarchy should exists between dimension tables.
It contains a fact table surrounded by dimension tables. If the dimensions are de-normalized, we say it is a star schema design.	It contains a fact table surrounded by dimension tables. If a dimension is normalized, we say it is a snow flaked design.
In star schema only one join establishes the relationship between the fact table and any one of the dimension tables.	In snow flake schema since there is relationship between the dimensions tables it has to do many joins to fetch the data.
Astar schema optimizes the performance by keeping queries simple and providing fast response time. All the information about the each level is stored in one row.	Snowflake schemas normalize dimensions to eliminated redundancy. The result is more complex queries and reduced query performance.
It is called a star schema because the diagram resembles a star.	It is called a snowflake schema because the diagram resembles a snowflake.

Fact constellation/ Galaxy schema

- Sophisticated applications may require multiple fact tables to *share* dimension tables
- It is a collection of multiple fact tables having some common dimension tables
- It can be viewed as a collection of several star schemas



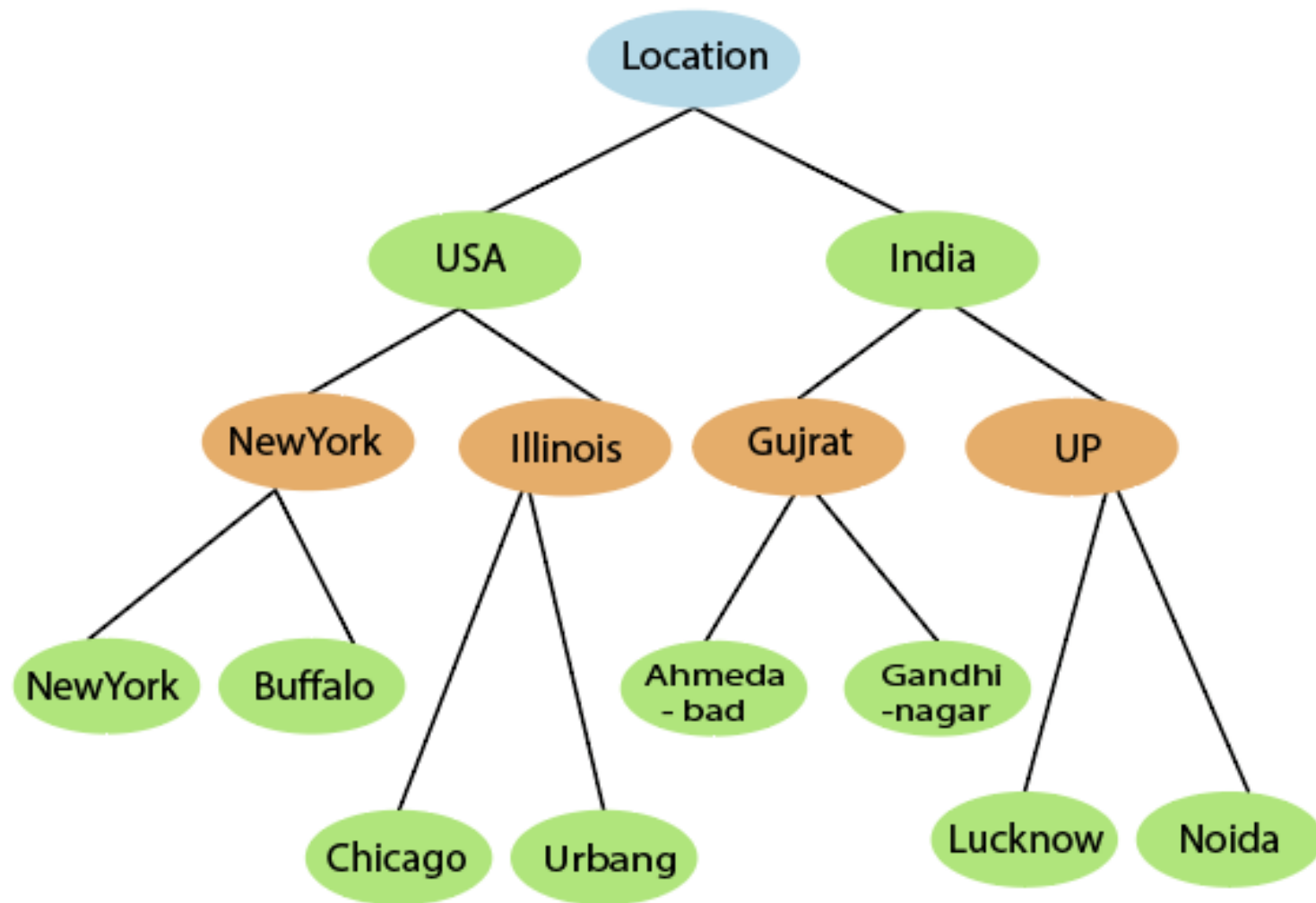
- Consider a Schema which specifies two fact tables, *sales* and *shipping*
- The *shipping* table has dimensions: *item*, *time*, *shipper*, *location* and two measures—*dollars cost* and *units shipped*
- The dimensions tables for *time*, *item*, and *location* are shared between the *sales* and *shipping* fact tables



Dimensions: The role of concept Hierarchies

- It defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts
- A **hierarchy** is an organizational structure in which items are ranked according to levels of importance
- Concept hierarchy formation: Concept Hierarchy recursively reduce the data by collecting and replacing low level concepts (such as numeric values for the attribute age) by higher level concepts (such as young, middle-aged, or senior)
- Can be explicitly specified by domain experts and/or datawarehouse designers

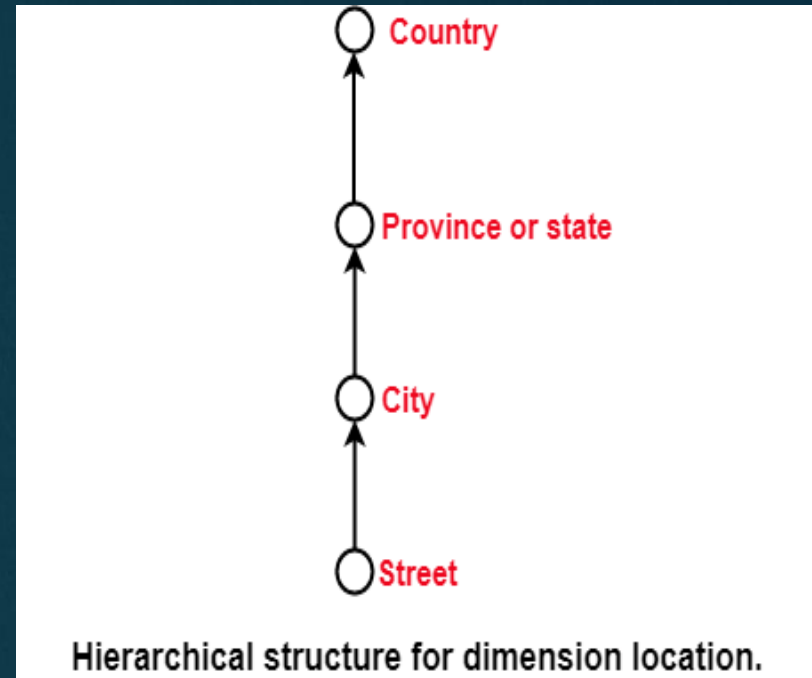
- Example: Consider a concept hierarchy for the dimension *location*
- City values for *location* can be mapped to the province or state to which it belongs
- The provinces and states can in turn be mapped to the country to which they belong
- Set of low-level concepts (i.e., cities) *are mapped to* higher-level, more general concepts (i.e., countries)



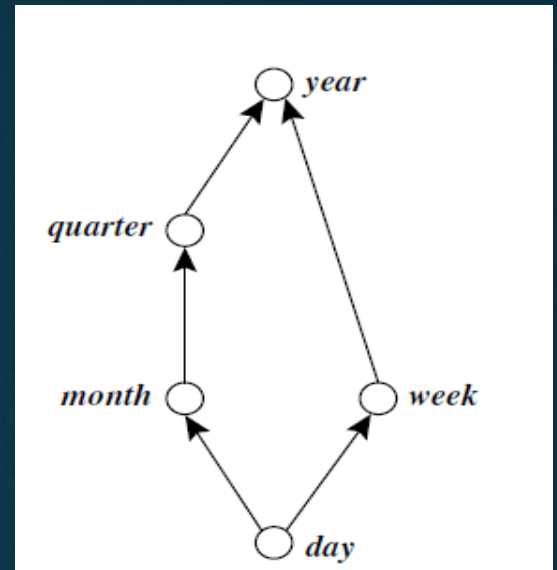
Concept hierarchy for the dimension location

Schema hierarchy

- Many concept hierarchies are implicit within the database schema
- This hierarchy may formally express semantic relation between attributes
- Example: dimension *location* is described by the attributes *number*, *street*, *city*, *province or state*, *zip code*, and *country*
- These attributes are related by a total order, forming a concept hierarchy such as “*street* < *city* < *province or state* < *country*” i.e street is conceptually lower than city which is lower than state which is lower than country

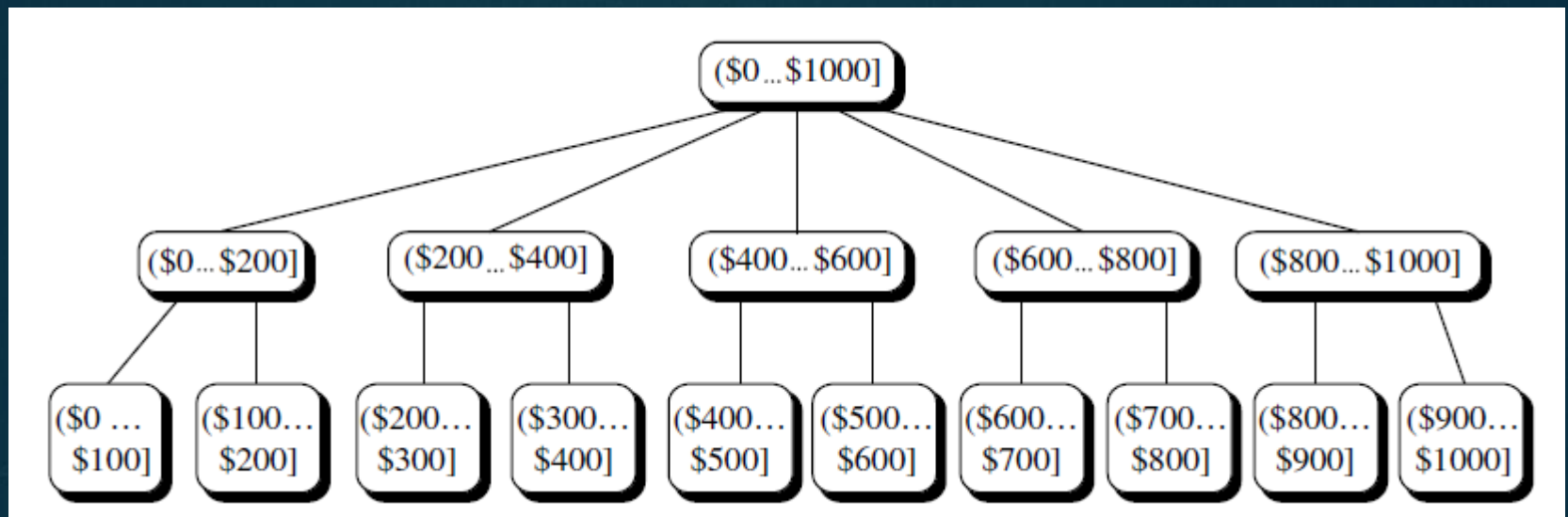


- The attributes of a dimension may be organized in a partial order, forming a lattice
- Partial order for the *time* dimension is described by the attributes *day*, *week*, *month*, *quarter*, and *year*
- These attributes are related by a Partial order, forming a concept hierarchy as
“*day* < {*month* < *quarter*; *week*} < *year*”
- A concept hierarchy that is a total or partial order among attributes in a database schema is called a **schema hierarchy**
- Concept hierarchies may be predefined in the data mining system

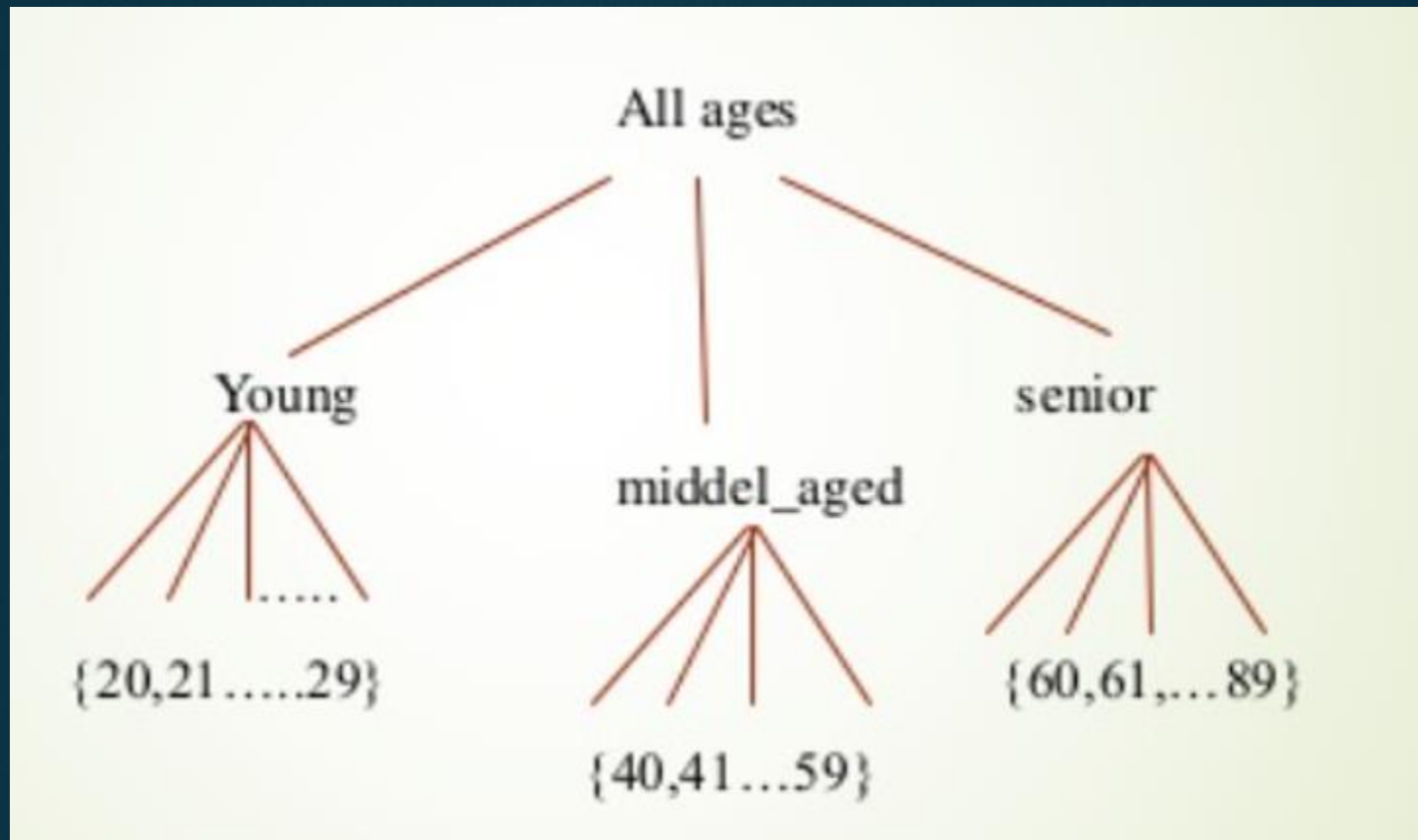


Set-grouping hierarchy

- Here, Concept hierarchies are defined by discretizing or grouping values for a given dimension or attribute
- A total or partial order can be defined among groups of values
- Example: for the dimension *price*, where an interval $(\$X \dots \$Y]$ denotes the range from $\$X$ (exclusive) to $\$Y$ (inclusive)



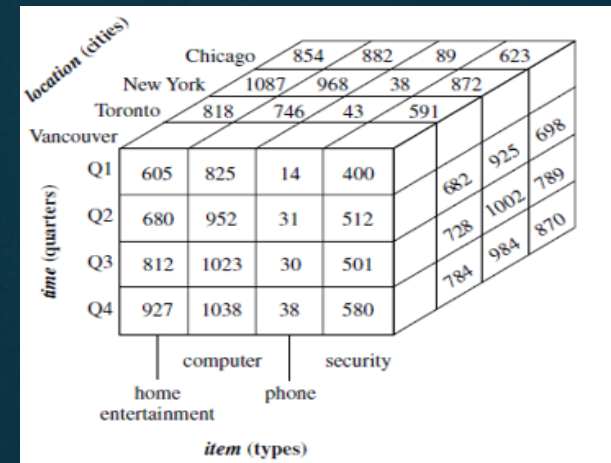
- Example2:



- There may be more than one concept hierarchy for a given attribute or dimension based on different user viewpoints
- Example: a user may prefer to organize *price* by defining ranges for *inexpensive*, *moderately priced*, and *expensive*
- Concept hierarchies may be provided manually by system users, domain experts, or knowledge engineers, or may be automatically generated based on statistical analysis of the data distribution
- Concept hierarchies allow data to be handled at varying levels of abstraction

Measures: Their Categorization and Computation

- *Multidimensional point* in the data cube space can be defined by a set of dimension–value pairs
 $\langle time = \text{“Q1”}, location = \text{“Vancouver”}, item = \text{“computer”} \rangle$
- A data cube **measure** is a numeric function that can be evaluated at each point in the data cube space
- A measure value is computed for a given point by aggregating the data corresponding to the respective dimension–value pairs defining the given point



- Measures can be organized into three categories based on the kind of aggregate functions used
 - Distributive
 - Algebraic
 - Holistic

Distributive Measures

- An aggregate function is *distributive* if it can be computed in a distributed manner
- Suppose the data are partitioned into n sets.
- We apply the function to each partition, resulting in n aggregate values.
- If the result derived by applying the function to the n aggregate values is the same as that derived by applying the function to the entire data set (without partitioning), the function can be computed in a distributed manner
- `sum()` can be computed for a data cube by first partitioning the cube into a set of subcubes, computing `sum()` for each subcube, and then summing up the counts obtained for each subcube
- A measure is *distributive* if it is obtained by applying a distributive aggregate function.
- Distributive measures can be computed efficiently because of the way the computation can be partitioned

Algebraic Measures

- An aggregate function is *algebraic* if it can be computed by an algebraic function with M arguments (where M is a bounded positive integer), each of which is obtained by applying a distributive aggregate function
- Example: `avg()` can be computed by `sum()/count()`, where both `sum()` and `count()` are distributive aggregate functions
- A measure is *algebraic* if it is obtained by applying an algebraic aggregate function

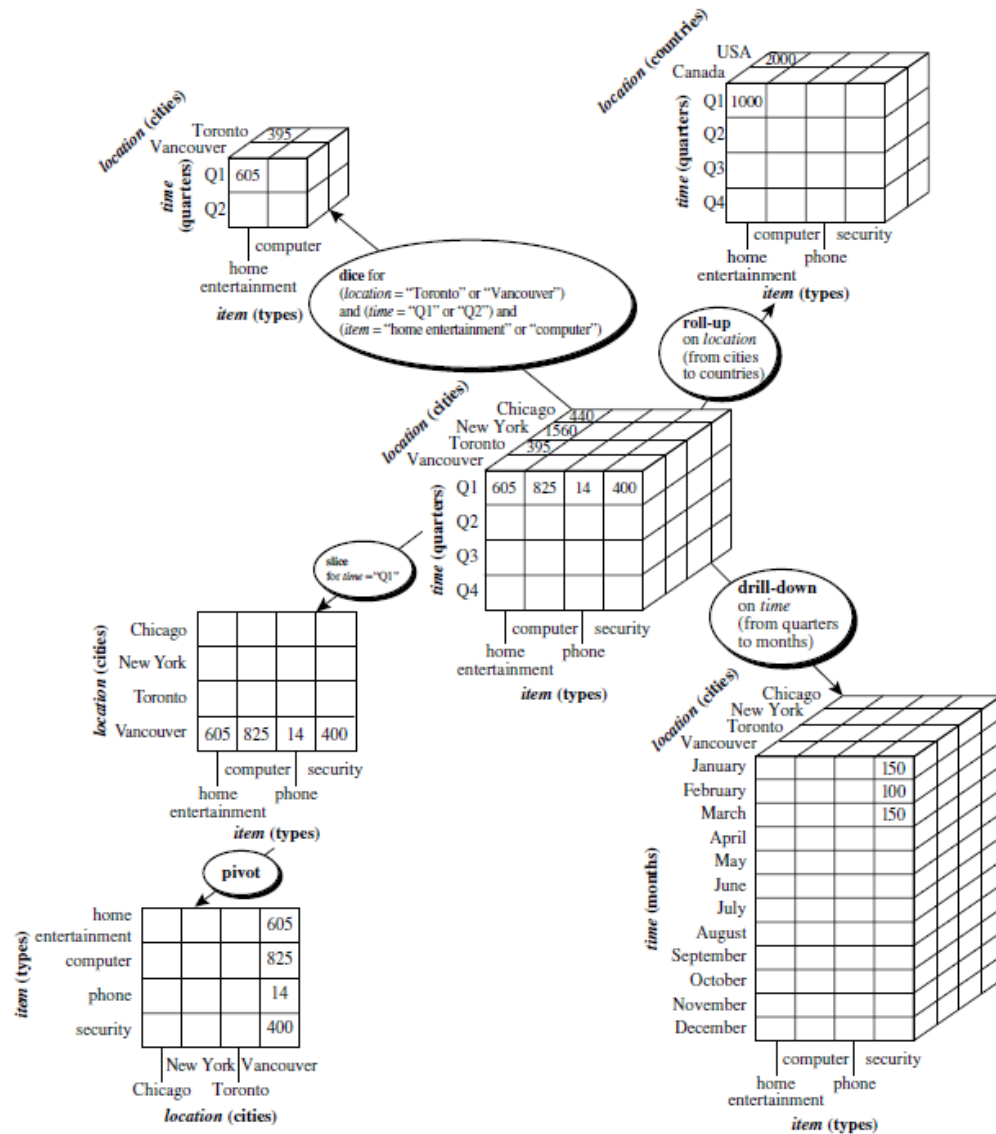
Holistic Measures

- An aggregate function is *holistic* if there is no constant bound on the storage size needed to describe a subaggregate.
- That is, there does not exist an algebraic function with M arguments (where M is a constant) that characterizes the computation
- Examples: `median()`, `mode()`, and `rank()` etc
- A measure is *holistic* if it is obtained by applying a holistic aggregate function

Typical OLAP Operations

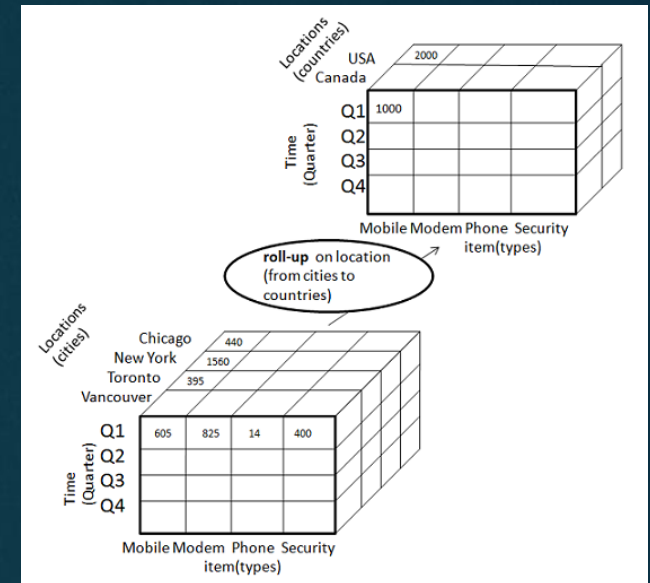
- In the multidimensional model, each dimension contains multiple levels of abstraction defined by concept hierarchies
- This organization provides users with the flexibility to view data from different perspectives
- A number of OLAP data cube operations exist to materialize these different views, allowing interactive querying and analysis of the data at hand

- Consider a data cube for *AllElectronics* sales
- Cube contains the dimensions: *location*, *time*, and *item*
- *location* is aggregated with respect to city values
- *time* is aggregated with respect to quarters
- *item* is aggregated with respect to item types
- Measure: *dollars sold* (in thousands).



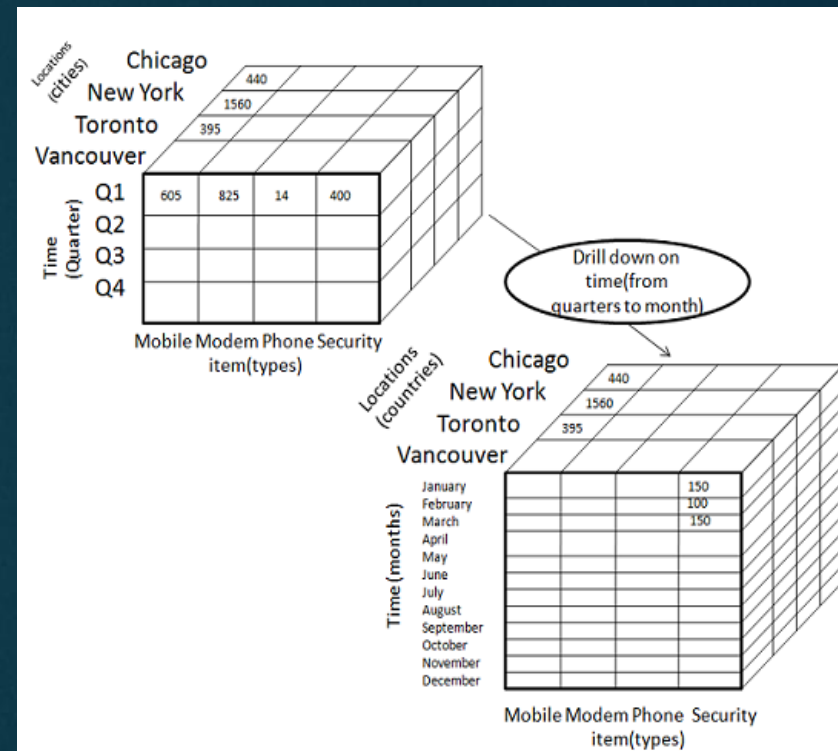
Roll-up/drill-up

- Performs aggregation on a data cube, either by **climbing up a concept hierarchy for a dimension** or by **dimension reduction**
- Concept hierarchy: total order “street < city < province or state < country”
- Roll-up operation aggregates the data by ascending the location hierarchy from the level of city to the level of country
- Rather than grouping the data by city, the resulting cube groups the data by country
- When roll-up is performed by dimension reduction, one or more dimensions are removed from the given cube
- Exmple: consider a sales data cube containing only the location and time dimensions
- Roll-up may be performed by removing, say, the time dimension, resulting in an aggregation of the total sales by location, rather than by location and by time



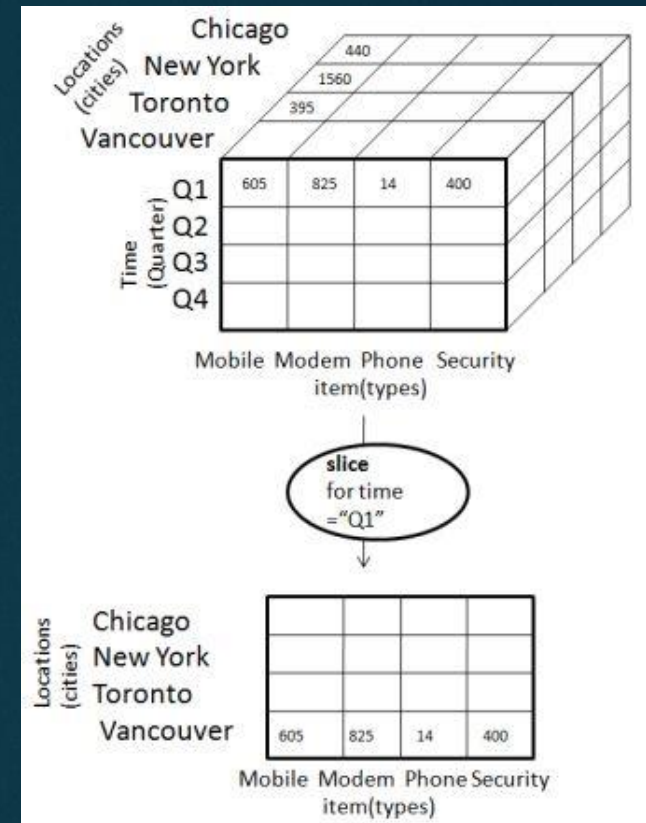
Drill-down

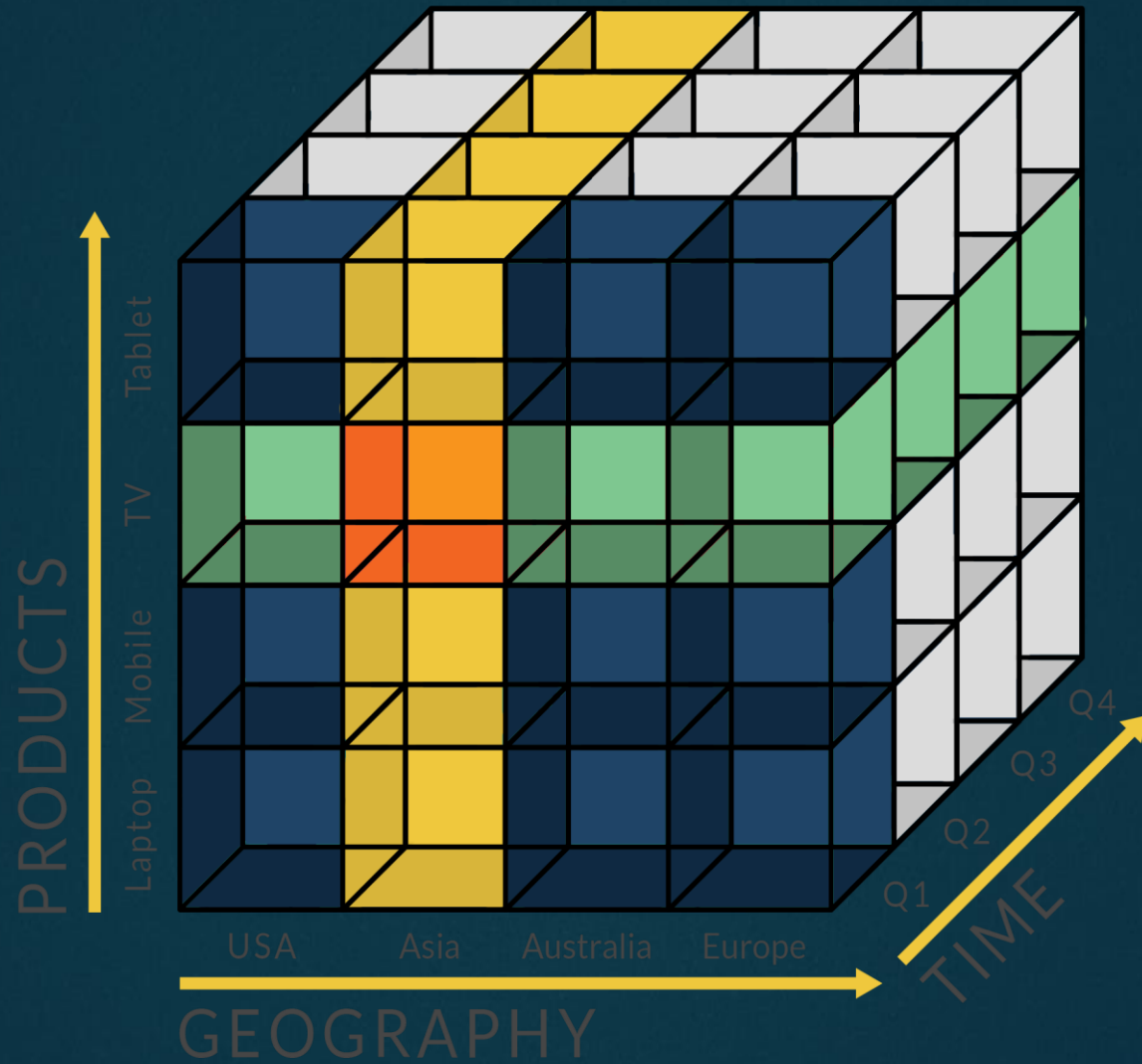
- Reverse of roll-up
- It navigates from less detailed data to more detailed data.
- Can be realized by either *stepping down a concept hierarchy for a dimension* or *introducing additional dimensions*
- Example: drill-down operation performed on the central cube by stepping down a concept hierarchy for *time* defined as
“*day < month < quarter < year.*”
- Drill-down occurs by descending the *time* hierarchy from the level of *quarter* to the more detailed level of *month*
- The resulting data cube details the total sales per month rather than summarizing them by quarter
- Because a drill-down adds more detail to the given data, it can also be performed by adding new dimensions to a cube



Slice

- This operation performs a selection on one dimension of the given cube, resulting in a subcube
- Sales data are selected from the central cube for the dimension *time* using the criterion *time* = “Q1”





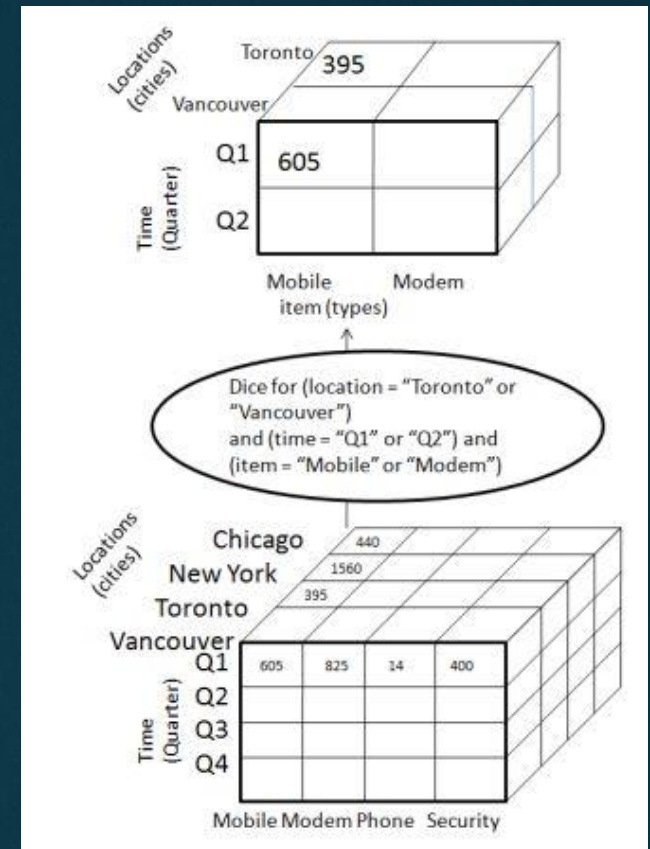
Dice

- This operation defines a subcube by performing a selection on two or more dimensions
- Dice operation on the central cube based on the following selection criteria that involve three dimensions:

(*location* = “Toronto” or “Vancouver”),

(*time* = “Q1” or “Q2”) and

(*item* = “home entertainment” or “computer”)



Pivot (rotate)

- It is a visualization operation that rotates the data axes in view to provide an alternative data presentation
- The *item* and *location* axes in a 2-D slice are rotated
- Examples: rotating the axes in a 3-D cube, transforming a 3-D cube into a series of 2-D planes

